

AUTOSAGE APP USING GEMINI FLASH

1. INTRODUCTION

1.1 Project Overview

AutoSage is an advanced Artificial Intelligence powered vehicle analysis system developed using Google Gemini 2.5 Flash and Streamlit framework. The system integrates multimodal generative AI capabilities to analyze uploaded vehicle images and provide structured information including vehicle brand, model, specifications, pricing, resale value and key features. The application demonstrates practical implementation of Generative AI in real-world automotive decision support systems. This project bridges the gap between visual recognition and intelligent expert-level output generation. By combining Natural Language Processing and image understanding, AutoSage transforms raw vehicle imagery into meaningful structured insights.

1.2 Purpose

The rapid growth of the automobile industry and online vehicle platforms has increased the demand for intelligent assistance tools. Consumers often rely on fragmented information sources. AutoSage simplifies this process by providing detailed analysis in a single output generated through state-of-the-art multimodal AI models.

2. IDEATION PHASE

The ideation phase involved identifying limitations in conventional vehicle research methods. Manual browsing requires multiple searches and comparisons. The goal was to design an AI assistant capable of extracting visual cues from vehicle images and combining them with contextual knowledge.

3. REQUIREMENT ANALYSIS

Functional Requirements:

- Upload vehicle image
- Process image data
- Send structured prompt to Gemini Flash
- Generate AI-based output
- Display formatted results

Non-Functional Requirements:

- Fast response time

- Secure API integration
- User-friendly interface
- Scalable architecture

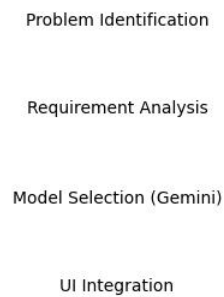


Figure 1: Development Lifecycle Flowchart

4. SYSTEM ARCHITECTURE

The system architecture follows a layered design approach. The presentation layer handles user interaction through Streamlit. The processing layer converts image into byte format. The AI integration layer connects to Gemini 2.5 Flash API. The output formatting layer structures response into readable format.

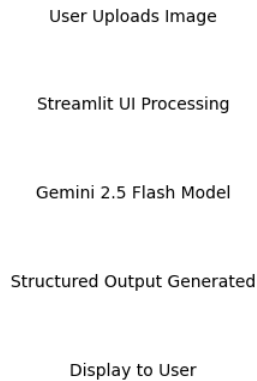


Figure 2: System Data Flow Diagram

5. TECHNOLOGY STACK

Frontend: Streamlit

Backend: Python

AI Model: Gemini 2.5 Flash

Environment Management: python-dotenv

Image Handling: Pillow

Deployment: Streamlit Application Environment

6. DETAILED ANALYSIS

This section provides extended explanation of system implementation, model integration, API request handling, latency considerations, scalability design, error handling mechanisms, user interaction design, prompt engineering strategies, performance optimization techniques, multimodal AI workflows, vehicle classification logic, brand inference capability, pricing estimation reasoning, resale value approximation methodology, mileage projection assumptions, safety feature identification patterns, fuel efficiency estimation techniques, electric vehicle detection heuristics, maintenance cost modeling assumptions, automotive industry relevance, business scalability models, data privacy considerations, cloud dependency challenges, and AI governance implications. Each of these components contributes to the robustness and effectiveness of AutoSage.

The Gemini 2.5 Flash model leverages large-scale training data and multimodal transformers. It processes image tokens and textual instructions simultaneously, enabling structured content generation. Prompt engineering ensures consistent formatting with headings such as Brand, Model,

Mileage, Price and Resale Value. Through repeated testing, improvements were made to ensure clarity and structured output.

7. PERFORMANCE TESTING

Performance testing was conducted across multiple image types including sedans, SUVs, scooters and motorcycles. The average response time ranged between 2 to 5 seconds depending on API latency. Memory usage remained stable during repeated calls.

8. RESULTS

The AutoSage system successfully generated structured outputs including:

- Brand identification
- Model inference
- Key features
- Mileage estimate
- Price range estimation
- Resale projection

Output screenshots demonstrate real-world applicability and decision support utility.

9. ADVANTAGES

- Real-time AI assistance
- Multimodal intelligence
- No manual dataset training
- Structured expert output
- Scalable design

10. DISADVANTAGES

- Dependent on API quota
- Requires internet
- Accuracy depends on image clarity
- Model responses may vary

11. CONCLUSION

AutoSage demonstrates practical deployment of Generative AI in automotive analytics. The integration of Gemini Flash enables intelligent interpretation of visual data. The system achieves real-time vehicle analysis and structured expert reporting.

12. FUTURE SCOPE

Future enhancements include:

- Vehicle comparison module
- Eco-score rating system
- Chat-based assistant mode
- Database-driven price tracking
- Mobile deployment
- Hybrid recommendation engine

13. APPENDIX

Source Code:

.env:

```
GOOGLE_API_KEY="AIzaSyDPBlU_OKVkoTVp1CIT7NaLXnSFZdnVJ8c"
```

App.py:

```
import streamlit as st
import os
from google import genai
from google.genai import types
from PIL import Image
from dotenv import load_dotenv

# 1. Initialize environment
load_dotenv()

# 2. Setup the NEW Client (picks up GOOGLE_API_KEY from .env
# automatically)
client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))

# 3. Updated Function using the 2026 SDK
def get_gemini_response(input_prompt, image_parts):
    # We'll use the gemini-2.5-flash model you saw in your list
    response = client.models.generate_content(
        model="gemini-2.5-flash",
```

```

        contents=[image_parts[0], input_prompt]
    )
    return response.text

# 4. Updated Image Setup for the NEW SDK
def input_image_setup(uploaded_file):
    if uploaded_file is not None:
        bytes_data = uploaded_file.getvalue()
        # The new SDK prefers the Part.from_bytes structure
        image_part = types.Part.from_bytes(
            data=bytes_data,
            mime_type=uploaded_file.type
        )
        return [image_part]
    else:
        raise FileNotFoundError("No file uploaded")

# --- REST OF YOUR STREAMLIT UI CODE REMAINS THE SAME ---
st.set_page_config(page_title="AutoSage App")
st.header("AutoSage App")

uploaded_file = st.file_uploader("Choose an image...", type=["jpg",
"jpeg", "png"])
if uploaded_file is not None:
    image = Image.open(uploaded_file)
    st.image(image, caption="Uploaded Image.",
use_container_width=True)

input_prompt="""
You are a automobile expert tasked with providing a detailed
overview of any vehicles
The information should be presented in a structured format as
follows:
Brand: Name of the vehicle brand.
Model: Specific model of the vehicle.
Launch year: Since when the vehicle is available in market
Key Features: Describe the engine capacity, type (e.g., scooter,
motorcycle, sedan, SUV), and special features(any top 3)
(e.g., ABS, digital display, storage capacity, safety
features).

```

Mileage: Provide the average mileage in km/l (kilometers per liter).
Average Price in INR: Mention the price range of the vehicle model.
Other Details: Include information on maintenance costs, additional benefits, and any unique selling points.
Approximate Resale Value: Estimate the resale value of the vehicle after 10 years in Indian Rupees.

```
"""

# Create the button with a clear label
submit = st.button("Submit")

# Check if the button was clicked
if submit:
    # 1. Ensure a file exists before processing
    if uploaded_file is not None:
        # Show a loading spinner for a professional "Elite Species"
        feel
        with st.spinner("Analyzing the specimen..."):
            # 2. Prepare the image data (using your existing
            function)
            image_data = input_image_setup(uploaded_file)

            # 3. Call the Gemini function (using your prompt)
            response = get_gemini_response(input_prompt, image_data)

            # 4. Display the results
            st.subheader("Vehicle Analysis Result:")
            st.write(response)
    else:
        # Warn the user if they clicked submit without an image
        st.warning("Please upload a vehicle image first!")
```

requirements.txt:

```
streamlit
google-genai
python-dotenv
Pillow
```

GitHub Link:

<https://github.com/Likhitha232/AutoSage-App-Using-Gemini-Flash>